

Homework 4

Due: 5:59pm ET Thursday October 8

This problem can be done in teams of up to 2 students.

Note: This homework is worth 60 points total. Bonus points from any problem cannot raise your total score above 60.

1 Window Type Classification [20 points]

In this problem, the goal is to classify window images into one of five categories: ‘Awning’, ‘Bay’, ‘Fixed’, ‘Horizontal Sliding’, and ‘Hung’. The training and test datasets can be accessed via the following links:

- https://drive.google.com/file/d/1-hWySWqodH3MirLV6PDJevwAssNFjYHG/view?usp=share_link
- https://drive.google.com/file/d/1l2xkLR4ZlvK9oPIJG--gA9hqNznvLaot/view?usp=share_link

To help you get started, starter code is available in `homework4_CNN_starter.ipynb`, which defines a baseline three-layer CNN, `SimpleCNN`, and resizes every image to 32×32 before training.

A note about this dataset, before you start: the training set is *imbalanced* by roughly $3\times$ across classes (the smallest class has a few hundred images, the largest over a thousand), and the images themselves come from mixed sources – some are clean product-catalog renders, others are real photos of installed windows shot at odd angles, in varying lighting. Report per-class accuracy in addition to overall accuracy so these effects don’t hide in an aggregate number; a class-weighted loss is sufficient to handle the imbalance itself (that is not the focus of this problem).

About the test set: `windows_test_set.zip` contains images only – no labels – so you cannot compute your own test accuracy. Wherever a task below asks you to report accuracy (Tasks 2 and 5), hold out a validation split from the *training* set and report accuracy on that split instead. Separately, you must submit a CSV file named `predictions.csv` with one row per image in `windows_test_set.zip`, in the format `filename,predicted_label` (using the class names in `class_names`, not raw indices). The TA will use this file to compute your model’s actual test accuracy, including whether you’ve met the bonus task’s 60% threshold.

Tasks:

1. **(2 points) Compute `fc1`’s input dimension by hand.** In this starter code, `SimpleCNN`’s `fc1` does *not* come with its input dimension pre-computed for you (nor does the matching `x.view(-1, ...)` call in `forward`) – you need to work it out yourself. Starting from the $32 \times 32 \times 3$ input, compute the output shape (channels $\times$ height $\times$ width) after `conv1`, its pooling layer, `conv2`, its pooling layer, `conv3`, and its pooling layer. Flatten the final shape into a single number, and use it to fill in both blanks.
2. **(3 points) Baseline.** `SimpleCNN` has a bug involving how its output interacts with `nn.CrossEntropyLoss()`, the criterion used to train it. Check the PyTorch documentation for what `nn.CrossEntropyLoss` expects as input, find the bug in `SimpleCNN.forward`, fix it, then train the corrected model. Report your overall accuracy and your per-class accuracy on a validation split you hold out from the training set.
3. **(3 points) Predict the receptive field.** Recall the receptive-field recurrence from lecture: for a layer with kernel size k and stride s ,

$$RF_{\text{out}} = RF_{\text{in}} + (k - 1) \cdot j_{\text{in}}, \quad j_{\text{out}} = j_{\text{in}} \cdot s,$$

where k and s are that layer’s own kernel size and stride, and j (the “jump”) is the distance, in input-image pixels, between two spatially adjacent units at that point in the network.

To use this, apply the recurrence one layer at a time, treating each convolution *and* each pooling operation as its own layer (so `SimpleCNN`’s `conv1` $\rightarrow$ `pool` $\rightarrow$ `conv2` $\rightarrow$ `pool` $\rightarrow$ `conv3` is 5 steps total). Start at the input with $RF = 1$ and $j = 1$ (adjacent input pixels are exactly 1 pixel apart), then, for each layer in order: read off its k and s , compute that layer’s RF_{out} and j_{out} using the layer’s own RF_{in} and j_{in} , and carry those values forward as the RF_{in} and j_{in} of the *next* layer.

Compute by hand the receptive field, with respect to the 32×32 input, of a single unit in the output of `conv3` itself (i.e., right after `conv3`’s own ReLU, but *before* its pooling layer). State clearly whether this receptive field covers the entire input image and, if not, how much of it it does cover.

4. **(3 points) Verify it empirically.** Pick a single unit in that same feature map (any channel, any spatial location). Using backpropagation (*not* training – just one backward pass), compute the gradient of that one unit’s activation with respect to every pixel of one input image, holding everything else fixed. Identify the input pixels with nonzero gradient and report the size of the resulting region. Confirm it matches your answer to Task 3 (small discrepancies from zero-padding at the border are expected – explain them if you see them).
5. **(4 points) Swap the head for Global Average Pooling.** Replace the code that flattens `conv3`’s output and feeds it through `fc1` and `fc2` with a Global Average Pooling operation (average each of the last conv layer’s channels down to a single number) followed by a *single* linear layer mapping directly to `num_classes` logits. Report this model’s total parameter count alongside the original’s, then retrain and report accuracy on your validation split.
6. **(5 points) Class Activation Maps.** Using your Task 5 (GAP) model, compute a Class Activation Map (CAM) for 3 correctly-classified test images, ideally from 3 different classes: take the last convolutional layer’s feature maps, form a weighted sum across channels using the final linear layer’s weights for the predicted class, upsample the result to the input image’s resolution, and overlay it on the original image (e.g., as a heatmap). Include all 3 overlays in your submission.
7. **(BONUS: 2 points) Push past 60% accuracy.** Using any combination of architecture changes, hyperparameter tuning, or changes to the training/optimization procedure, try to get your best model’s test accuracy – as computed by the TA from your submitted `predictions.csv` – to at least 60%. Briefly describe what you changed and why.

2 Comparing Vanilla RNN with Variants in Sequence Modeling [20 points]

You will implement and train three different neural networks for sequence modeling: a Vanilla RNN (a simple RNN with shared weights), and two variants of a NN with a similar architecture to the Vanilla RNN but which do not share weights. You will compare their performance on a sequence prediction task and analyze the differences between them.

Prediction task: this is a many-to-one regression task, i.e., a sequence of inputs is used for predicting a single output. In particular, the i -th input sequence $\mathcal{X}^{(i)} = (\mathbf{x}_1^{(i)}, \dots, \mathbf{x}_{\ell_i}^{(i)})$ has length ℓ_i , where $\mathbf{x}_j^{(i)} \in \mathbb{R}^{10}$; the i -th output is a scalar $y^{(i)} \in \mathbb{R}$.

Dataset: You will use a synthetic dataset containing sequences of variable lengths stored in the zip file: https://drive.google.com/file/d/1ER6ljxw8RzTXJJ_zdY8IsTYfyANEILUN/view?usp=share_link. Each sequence consists of input features and corresponding target values. The sequences are generated such

that they represent a time-dependent process. Note that ℓ_i may be different than ℓ_j for $i \neq j$. So the (pickled) numpy object $\mathbf{X}$ is actually a list of sequences.

Tasks:

1. (4 points) Implement a Vanilla RNN: Implement a Vanilla RNN architecture (needless to say, weights are shared across time steps). A pytorch starter code `homework4_RNN_starter.ipynb` is provided on Canvas. **Important: You are not allowed to use an RNN layer implementation from any library.**
2. (4 points) Implement a NN with Sequences Truncated to the Same Length: Implement a NN where sequences are truncated to have the same length before training. In other words, if the shortest sequence in the dataset has length L , all sequences should be truncated to length L before training.
3. (4 points) Implement a NN with Sequences Padded to the Same Length: Implement another variant of NN where sequences are padded to have the same length before training. Use appropriate padding techniques to ensure that all sequences have the same length, and implement a mechanism to ignore the padding when computing loss and predictions.
4. (4 points) **Train all three models, then predict before you evaluate.** Train all three models (Vanilla RNN, Truncated NN, Padded NN) on the provided dataset, using mean squared error (MSE) as your loss function, for a fixed number of epochs; monitor and record the training and validation loss curves for each model. Then, before looking at test-set performance, predict the *ranking* of the three models by overall test MSE (best to worst). Justify your prediction using the actual mechanics of the three architectures (shared weights vs. fixed-length truncation vs. fixed-length padding) – not just intuition.
5. (4 points) **Reconcile and analyze.** Evaluate all three trained models on the test set and report the overall test MSE for each. Compare the actual ranking to your Task 4 prediction. Where you were right, explain why using the models' mechanics; where you were surprised, propose a concrete explanation. Discuss the trade-offs between shared weights, truncation, and padding for modeling sequences of varying length.

Additional Information:

You can choose the specific hyperparameters for your models, such as the number of hidden units, learning rate, and batch size. Feel free to use any deep learning framework or library you are comfortable with, and provide clear code documentation. Note: Be sure to clearly explain your implementation, provide code comments, and present your results in a well-organized manner in the report.

3 Fine-tune a DistilBERT model [20 points + 2 bonus points]

In this project, you will first train a classification head using a pre-trained DistilBERT model on a dataset of social media tweets to classify tweets as containing medical information or not. You are provided with a dataset of social media tweets, where each tweet is labeled as either containing medical information (class 1) or not containing medical information (class 0).

A pytorch starter code `homework4_finetuning_starter.ipynb` is provided on Canvas. It already includes – fully working, no changes needed – loading the dataset, a stratified train/val/test split, tokenization, and a generic training/evaluation loop; this lets you spend your time on the modeling questions below rather than on data plumbing.

Useful documentation for this question can be found here:

<https://huggingface.co/docs/transformers/index>

<https://huggingface.co/docs/transformers/training>

https://huggingface.co/docs/transformers/tasks/sequence_classification

Tasks:

1. (3 points) Tensor shapes through your own model.

Figure 1 shows one self-attention sub-layer of a transformer. Rather than using made-up numbers, use the DistilBERT model you are about to fine-tune below: inspect `model.config` to read off the hidden size (`dim`), number of attention heads (`n_heads`), and number of layers (`n_layers`); use the batch size B (`BATCH_SIZE`) and the max_length L (`MAX_LENGTH`) set in the starter code. These shapes come from the model's fixed architecture, not from training, so you can do this before any of the tasks below. Using these values, compute, showing your work:

- The shape of Q, K, and V for a single head
- The shape of Q, K, and V (combined, before splitting into heads)
- The shape of the attention map (output of softmax)
- The shape of Dropout-1's output
- The shape of Output (after concatenating all heads and applying the output projection)
- The total number of parameters in *one* self-attention sub-layer as pictured (i.e., just the Q, K, V, and Output projections and their biases – not the whole transformer block), expressed as a formula in terms of the hidden size, and then as a number for your specific model.

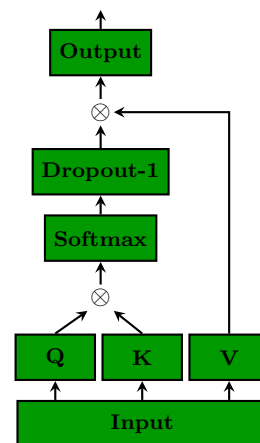


Figure 1: One self-attention sub-layer of a transformer, input at the bottom.

- (3 points) Before you freeze anything.** The starter code loads the pretrained `DistilBertForSequenceClassification` model for you. Inspect its structure via `model.named_parameters()` (or `print(model)`). (a) Compute the total number of parameters in the model, and separately the number of parameters belonging only to the classification head (`pre_classifier` and `classifier`) – not the DistilBERT backbone. (b) Based on these two counts, what fraction of the model's parameters will actually receive gradient updates if you freeze the backbone? (c) In 1–2 sentences, predict how this should affect training speed and memory use compared to fine-tuning the whole model – you will check this prediction in Task 5.
- (5 points) Grid search with a frozen backbone.** Implement grid search for at least one hyperparameter (e.g., learning rate): for each setting, start from a *freshly loaded* pretrained model, freeze every parameter in the DistilBERT backbone (`requires_grad=False`) so that only the classification head you identified in Task 2 trains, then train on the training set and evaluate F1-macro on the validation set. Report the F1-macro for every setting you tried and identify the best one. **Important:** re-loading a fresh pretrained model for each setting is not optional – reusing a model that already trained under a previous setting will contaminate your comparison.
- (3 points) Final frozen-head training.** Run the final training on train+val with the best hyperparameters from Task 3 (classification head only, backbone still frozen). This is your *frozen-head* model – keep it and its F1-macro score, you will need them for Task 5.
- (3 points) Is freezing the backbone actually worth it?** Repeat the final training from Task 4, but this time *unfreeze every parameter* (`requires_grad=True` for the whole model, not just the head) and fine-tune the whole model, using a smaller learning rate suitable for full fine-tuning (e.g., 2×10^{-5} instead of a head-only-scale learning rate). Report: (a) the F1-macro of this fully fine-tuned model vs. your frozen-head model from Task 4, and (b) the wall-clock training time of each, and how that ratio compares to your Task 2(b)/(c) prediction. In 2–3 sentences, discuss whether the accuracy gained (if

any) was worth the extra compute, and connect this to what freezing the backbone actually does to the optimization problem.

6. **(3 points) Interpreting F1-macro.** While running your Task 3 grid search, you may have noticed that some hyperparameter settings cause the model to always predict the same class (e.g., always predicting the negative class) regardless of input – a common failure mode on this imbalanced dataset. Identify such a setting among your grid-search results if one occurred (if none did, pick your worst-performing setting instead), and report its F1-macro. Compute by hand what the precision and recall for *each* class must be for a model that always predicts a single label, and use that to explain why F1-macro is low in this case even though overall accuracy can look reasonable. In 2–3 sentences, discuss why F1-macro (rather than accuracy) is the right metric to watch on this dataset.
7. (BONUS: 2 points) Apply a principled change to your code in order to achieve $F1\text{-macro} > 0.50$. Explain what you did and why you did it.

Submission

Submit one ZIP file that includes your jupyter notebooks. Answers to the non-coding questions must be written in designated the markdown cells. For Problem 1, also submit your `predictions.csv` file (see Problem 1’s task list) as a separate file alongside the PDF.